

CSE2430 - Operating Systems

Homework Processes and Scheduling

Turn-in Instructions

No hand-in required. Questions are meant for self-directed exam preparation and can be discussed in the midterm Q&A session.

1 Best and Worst TAT

Consider a First-come First-served scheduler as presented in class and a task queue with n tasks of length 1 and one task of length 10. Find a formula (as a function of n) for the average turnaround time of a task for

- a) the best case schedule
- b) the worst case schedule.

Create a graph for the best case and worst case turnaround time for 1 to 25 tasks. Explain your result.

2 Concurrency

Will the following program given in pseudo code always have the same output?

Explain how your hardware and choice in scheduler can influence the outcome of the program.

```
int pid = 0
pid = fork()
if(pid)
    print "parent"
else
    print "child"
```

3 Number of processes

Let's consider the following program.

```
int main() {
    for (int i = 0; i < 8; i++) {
        pid_t pid = fork();
        if (pid != 0) {
            wait(NULL);
            printf("%d\n", i);
            exit(0);
        }
    }
    return 0;
}
```

- Explain what the `wait` system call does (in general).
- How many processes (including the original one) does this program create? Justify your answer.
- What is the output of this program? Justify your answer.

4 Scheduling

Five processes *A*, *B*, *C*, *D*, and *E* arrive at the indicated moments in the given order with the following CPU burst (task length) and priority values:

	CPU Burst	Priority	Arrival time
A	2	1	0
B	3	2	0
C	1	5	1
D	2	4	4
E	3	3	5

Like in Linux, a lower priority value means higher priority.

Fill in the **turnaround time** of each process for the different scheduling policies.

For policies working with a time quantum, the scheduler switches immediately if a task finished before the time slice is exhausted.

Scheduling Policy	Response Time				
	A	B	C	D	E
Round-Robin (time quantum = 3)					
Preemptive, Shortest Job First					
Priority-based					

5 `clock_gettime`

Educate yourself (e.g., using man pages or programming examples on the Internet) about the use of `clock_gettime` with `CLOCK_MONOTONIC` option for getting time values with usually nanosecond resolution. Use this clock in a small C program to measure the overhead of

- a) a simple system call like `SYS_getpid`
- b) a function call.

Explain what you find.

Keep in mind that due to the resolution of the clock you need to measure over a reasonably large number of calls. Compile your code without optimization (`-O0`) to avoid that the compiler optimizes critical parts of your code away.